AdvR 14 – Evaluation

Evaluation Basics

Evaluation Overview

Quotation lets users selectively evaluate (unquote). **Evaluation** lets developers run quoted code in custom environments.
Together with quosures and data masks, they form **tidy evaluation**:

Pillar	Purpose
Quasiquotation	Build code from templates (`!!`, `!!!`)
Quosures	Bundle expression + environment
Data masks	Evaluate with data frame columns as variables

eval() Basics

`eval()` takes an expression and an environment:

```
x <- 10; y <- 2
eval(expr(x + y))
```

```
#> [1] 12
```

Override the environment to change what symbols mean:

```
eval(expr(x + y), env(x = 1000))
```

```
#> [1] 1002
```

Caution: `eval()` doesn't quote its first argument:

```
eval(print(x + 1), env(x = 1000))   # x+1 evaluated FIRST
```

```
#> [1] 11
```

```
#> [1] 11
```

```
eval(expr(print(x + 1)), env(x = 1000))  # correct
```

```
#> [1] 1001
```

Application: `local()`

`local()` evaluates code in a temporary environment:

```
foo <- local({ x <- 10; y <- 200; x + y })
foo
```

```
#> [1] 210
```

```
x   # not found -- x was local
```

```
#> [1] 10
```

Reimplemented in a few lines:

```
local2 <- function(expr) {
  env <- env(caller_env())
  eval(enexpr(expr), env)
}
local2({ a <- 10; b <- 200; a + b })
```

```
#> [1] 210
```

Application: `source()`

Combine `eval()` with `parse_exprs()` to execute a file:

```
source2 <- function(path, env = caller_env()) {
  file <- paste(readLines(path, warn = FALSE),
    collapse = "\n")
  exprs <- parse_exprs(file)
  res <- NULL
  for (i in seq_along(exprs))
    res <- eval(exprs[[i]], env)
  invisible(res)
}
```

Pattern: read → parse → evaluate each expression in turn.

Exercises (Eval Basics)

1. Predict the results:

```
eval(expr(eval(expr(eval(expr(2 + 2))))))
eval(eval(expr(eval(expr(eval(expr(2 + 2)))))))
expr(eval(expr(eval(expr(eval(expr(2 + 2)))))))
```

2. Re-implement `get()` using `sym()` and `eval()`:

```
get2 <- function(name, env) {}
```

Solutions (Eval Basics)

1. First: `4` (eval peels each layer). Second: `4` (same). Third: returns the **expression** `eval(expr(eval(expr(eval(expr(2 + 2))))))` because the outer `expr()` quotes everything.
2.

```
get2 <- function(name, env = caller_env()) {
  eval(sym(name), env)
}
get2("mean")
```

```
#> function (x, ...)
#> UseMethod("mean")
#> <bytecode: 0x58baf2400590>
#> ... [output truncated]
```

Quosures

What Are Quosures?

A **quosure** = expression + environment. Bundles captured code with **where** it was written:

```
foo <- function(x) enquo(x)
foo(a + b)
```

```
#> <quosure>
#> expr: ^a + b
#> env:  global
#> ... [output truncated]
```

Three ways to create: `enquo()` (most common), `quo()`, `new_quosure()`:

```
new_quosure(expr(x + y), env(x = 1, y = 10))
```

```
#> <quosure>
#> expr: ^x + y
#> env:  0x58baf1469318
#> ... [output truncated]
```

Evaluating Quosures

`eval_tidy()` evaluates a quosure in its bundled environment:

```
q1 <- new_quosure(expr(x + y), env(x = 1, y = 10))
eval_tidy(q1)
```

```
#> [1] 11
```

Essential for `...` where each argument may have a **different** environment:

```
f <- function(...) { x <- 1; g(..., f = x) }
g <- function(...) enquos(...)
x <- 0
qs <- f(global = x)
map_dbl(qs, eval_tidy)
```

```
#> global      f
#>      0      1
```

Quosures Under the Hood

Quosures are a subclass of **formulas** (~):

```
q4 <- new_quosure(expr(x + y + z))
class(q4)
```

```
#> [1] "quosure" "formula"
```

```
is_call(q4)
```

```
#> [1] TRUE
```

Extract parts with `get_expr()` and `get_env()`:

```
get_expr(q4)
```

```
#> x + y + z
```

```
get_env(q4)
```

```
#> <environment: R_GlobalEnv>
```

Exercises (Quosures)

1. Predict the result of evaluating each quosure:

```
q1 <- new_quosure(expr(x), env(x = 1))
q2 <- new_quosure(expr(x + !!q1), env(x = 10))
q3 <- new_quosure(expr(x + !!q2), env(x = 100))
eval_tidy(q3)
```

Solutions (Quosures)

1. q1 evaluates to 1. q2 = `x + q1` in env where `x = 10`, so $10 + 1 = 11$. q3 = `x + q2` in env where `x = 100`, so $100 + 11 = \mathbf{111}$.

```
q1 <- new_quosure(expr(x), env(x = 1))
q2 <- new_quosure(expr(x + !!q1), env(x = 10))
q3 <- new_quosure(expr(x + !!q2), env(x = 100))
eval_tidy(q3)
```

```
#> [1] 111
```

Data Masks

Data Mask Basics

A **data mask** = data frame where `eval_tidy()` looks for variables first:

```
q1 <- new_quosure(expr(x * y), env(x = 100))
df <- data.frame(y = 1:5)
eval_tidy(q1, df)
```

```
#> [1] 100 200 300 400 500
```

Wrap it into a `with()`-like function:

```
with2 <- function(data, expr) {
  expr <- enquo(expr)
  eval_tidy(expr, data)
}
x <- 100
with2(df, x * y)
```

```
#> [1] 100 200 300 400 500
```

Pronouns: `.data` and `.env`

Data masks create **ambiguity** — is `x` from the data or the environment?
Resolve with **pronouns**:

- `.data$x` — always from the data mask
- `.env$x` — always from the environment

```
x <- 1
df <- data.frame(x = 2)
c(with2(df, .data$x), with2(df, .env$x))
```

```
#> [1] 2 1
```

```
with2(df, .data$y)   # error: y not in data
```

```
#> Error in `.data$y`:
#> ! Column `y` not found in `.data`.
```

Application: `subset2()`

Tidy eval version of `base::subset()`:

```
subset2 <- function(data, rows) {
  rows <- enquo(rows)
  rows_val <- eval_tidy(rows, data)
  stopifnot(is.logical(rows_val))
  data[rows_val, , drop = FALSE]
}
sample_df <- data.frame(a = 1:5, b = 5:1,
  c = c(5, 3, 1, 4, 1))
subset2(sample_df, b == c)
```

```
#>   a b c
#> 1 1 5 5
#> 5 5 1 1
```

Application: `transform2()`

Tidy eval version of `base::transform()`:

```
transform2 <- function(.data, ...) {
  dots <- enquos(...)
  for (i in seq_along(dots)) {
    name <- names(dots)[[i]]
    .data[[name]] <- eval_tidy(dots[[i]], .data)
  }
  .data
}
df <- data.frame(x = c(2, 3, 1), y = runif(3))
transform2(df, x2 = x * 2, y = -y)
```

```
#>   x          y x2
#> 1 2 -0.2863975  4
#> 2 3 -0.6885851  6
#> 3 1 -0.8730037  2
```

Uses a for loop (not `map()`) so each column sees previous columns.

Tidy Evaluation in Practice

Quote and Unquote Pattern

When wrapping a quoting function, **quote** the user's input, then **unquote** when passing it on:

```
subsample <- function(df, cond, n = nrow(df)) {
  cond <- enquo(cond)
  df <- subset2(df, !!cond)
  df[sample(nrow(df), n, replace = TRUE), ,
    drop = FALSE]
}
df <- data.frame(x = c(1,1,1,2,2), y = 1:5)
subsample(df, x == 1)
```

```
#>     x y
#> 1   1 1
#> 1.1 1 1
#> ... [output truncated]
```

Pattern: `enquo()` → `!!` when calling downstream.

Handling Ambiguity with Pronouns

Without pronouns, bugs arise silently:

```
threshold_x <- function(df, val) {
  subset2(df, x >= val)
}
has_val <- data.frame(x = 1:3, val = 9:11)
threshold_x(has_val, 2)  # uses df$val, not val=2!
```

```
#> [1] x   val
#> <0 rows> (or 0-length row.names)
```

Fix with `.data` and `.env`:

```
threshold_x <- function(df, val) {
  subset2(df, .data$x >= .env$val)
}
threshold_x(has_val, 2)  # correct
```

```
#>   x val
#> 2 2  10
#> 3 3  11
#> ... [output truncated]
```

Rule: `.data$` for data columns; `.env$` (or `!!`) for function arguments.

Exercises (Tidy Eval)

1. Why use a for loop in `transform2()` instead of `map()`? Consider `transform2(df, x = x * 2, x = x * 2)`.
2. What does this alternative `threshold_var()` do differently?

```
threshold_var <- function(df, var, val) {
  var <- as_string(ensym(var))
  subset2(df, .data[[var]] >= !!val)
}
```

Solutions (Tidy Eval)

1. A for loop lets each column see **previous modifications**. `transform2(df, x = x * 2, x = x * 2)` quadruples `x` with a for loop (first pass doubles, second doubles again). With `map()`, both would see the original `x`, so `x` would only double.
2. `ensym()` captures `var` as a symbol, `as_string()` converts to a character. `.data[[var]]` uses string indexing — more robust than `.data$var` which would literally look for a column named `var`. `!!val` unquotes `val` early, avoiding ambiguity if `val` is a column name.